Exercise Set 1

Problem 1

Task a

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


df = pd.read_csv("train.csv")
```

```
df.drop(columns=['id', 'partlybad'], inplace=True)
```

Task b

```
df[['T84.mean', 'UV_A.mean', 'CS.mean']].describe()
```

	T84.mean	UV_A.mean	CS.mean
count	450.000000	450.000000	450.000000
mean	6.439594	10.797330	0.003083
std	9.827282	6.626678	0.002246
min	−23.627710	0.438965	0.000343
25%	−0.733160	4.304971	0.001436
50%	7.661622	11.506937	0.002548
75%	14.118346	16.573188	0.004119
max	27.001804	22.500037	0.013706

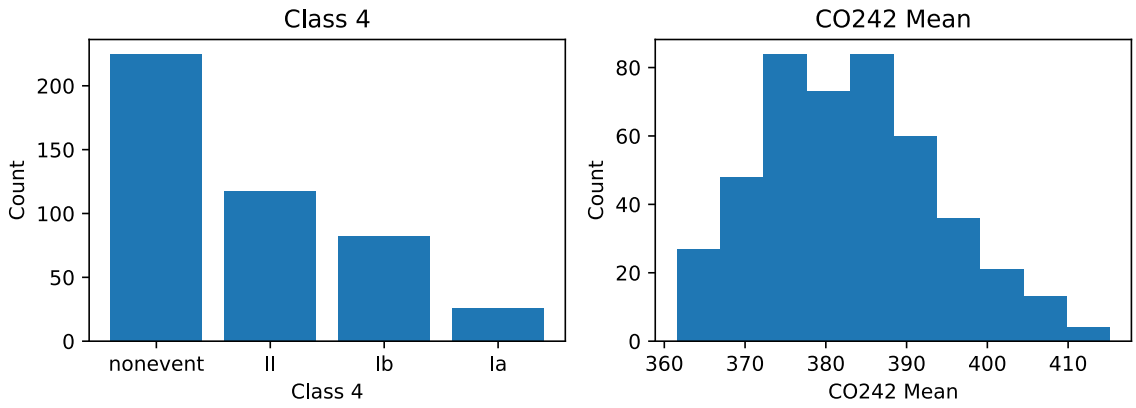
Task c

```
arr = df['T84.mean'].values
mean = arr.mean()
std = arr.std()
print(f"Mean: {mean}, Std: {std}")
```

```
Mean: 6.439594405677285, Std: 9.816356767300478
```

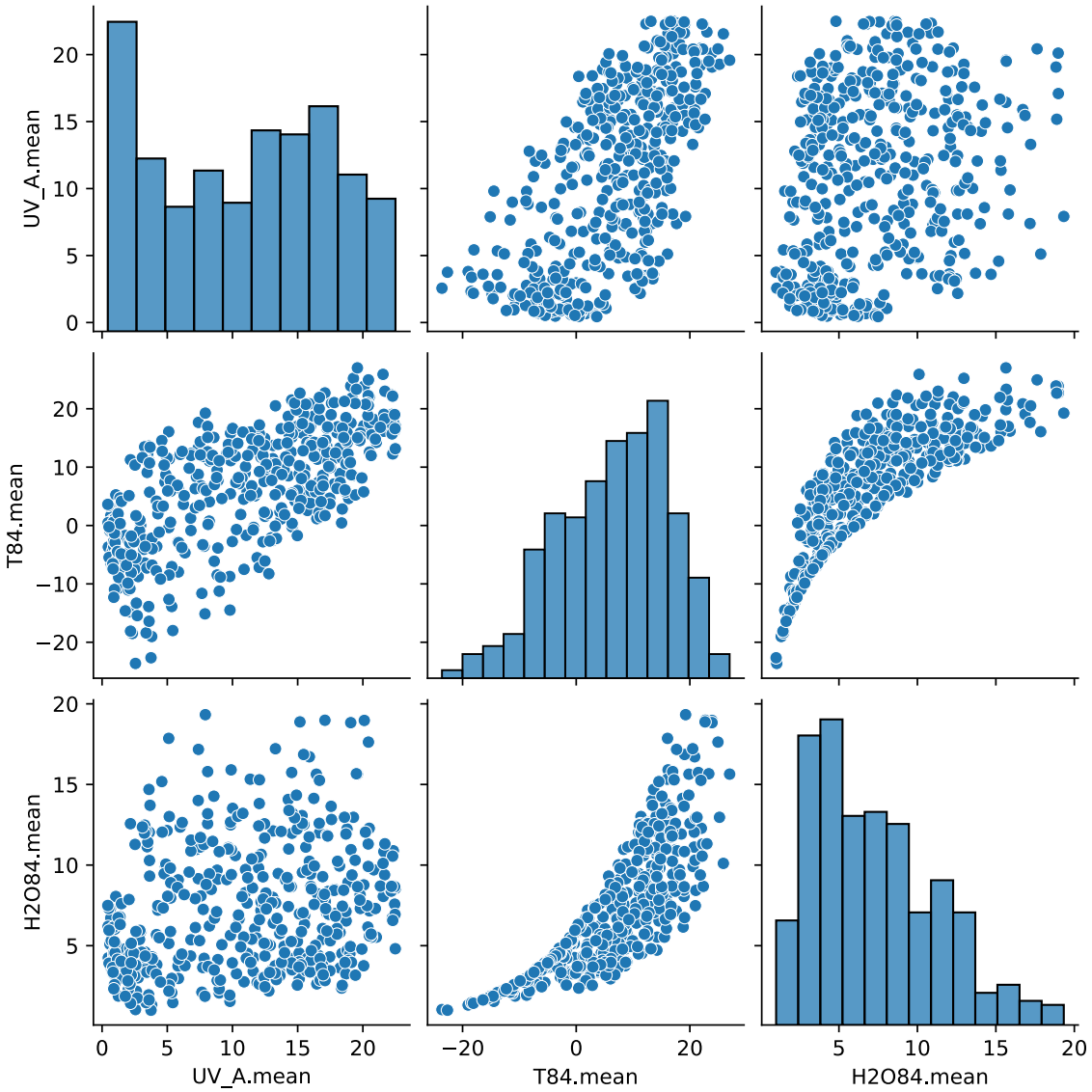
Task d

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3))

axes[0].bar(df['class4'].value_counts().index,
df['class4'].value_counts().values)
axes[0].set_title('Class 4')
axes[0].set_xlabel('Class 4')
axes[0].set_ylabel('Count')

axes[1].hist(df['CO242.mean'])
axes[1].set_title('CO242 Mean')
axes[1].set_xlabel('CO242 Mean')
axes[1].set_ylabel('Count')

plt.tight_layout()
plt.show()
```



Task e

```
sns.pairplot(df[['UV_A.mean', 'T84.mean', 'H2O84.mean']])
plt.show()
```



Task f

```
most_common_class = df['class4'].value_counts().idxmax()
p = (len(df) - df['class4'].value_counts()['nonevent']) / len(df)

test = pd.read_csv("test.csv")
submission = pd.DataFrame({'id': test['id'], 'class4': most_common_class, 'p':
p})
submission.to_csv("submission.csv", index=False)
```

Problem 2

Task a

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score

train_syn = pd.read_csv("train_syn.csv")
valid_syn = pd.read_csv("valid_syn.csv")
test_syn = pd.read_csv("test_syn.csv")

degrees = np.arange(0, 9)
models = {}

train_loss = []
valid_loss = []
test_loss = []
test_trva_loss = []
cv_loss = []
```

```
X_train = train_syn.drop(columns=['y'])
y_train = train_syn['y']
X_valid = valid_syn.drop(columns=['y'])
y_valid = valid_syn['y']
X_test = test_syn.drop(columns=['y'])
y_test = test_syn['y']
X_trva = pd.concat([X_train, X_valid], axis=0)
y_trva = pd.concat([y_train, y_valid], axis=0)
```

```
def make_poly_ols(degree: int) -> Pipeline:
    include_bias = True if degree == 0 else False
    return Pipeline([
        ("poly", PolynomialFeatures(degree=degree,
include_bias=include_bias)),
        ("scaler", StandardScaler()),
        ("ols", LinearRegression())
    ])

def compute_loss(degree: int, X_train: pd.DataFrame, y_train: pd.Series,
X_test: pd.DataFrame, y_test: pd.Series) -> float:
    model = make_poly_ols(degree)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    return np.mean((y_test - y_pred) ** 2)
```

```
def cv_score_model(model: Pipeline, X: pd.DataFrame, y: pd.Series, cv: int =
10) -> float:
    kf = KFold(n_splits=cv, shuffle=True, random_state=42)
    scores = cross_val_score(model, X, y, scoring='neg_mean_squared_error',
cv=kf)
    return -scores.mean()

def compute_all_losses(degree: int):
    train_loss.append(compute_loss(degree, X_train, y_train, X_train,
y_train))
    valid_loss.append(compute_loss(degree, X_train, y_train, X_valid,
y_valid))
    test_loss.append(compute_loss(degree, X_train, y_train, X_test, y_test))
    test_trva_loss.append(compute_loss(degree, X_trva, y_trva, X_test,
y_test))

    cv_loss.append(cv_score_model(make_poly_ols(degree), X_trva, y_trva))
    model = make_poly_ols(degree)
    model.fit(X_trva, y_trva)
    models[degree] = model
```

```
for degree in degrees:
    compute_all_losses(degree)

results_df = pd.DataFrame({
    'Degree': degrees,
    'Train': train_loss,
    'Valid': valid_loss,
    'Test': test_loss,
    'TRVA': test_trva_loss,
    'CV': cv_loss
}).set_index('Degree')
print(results_df)
```

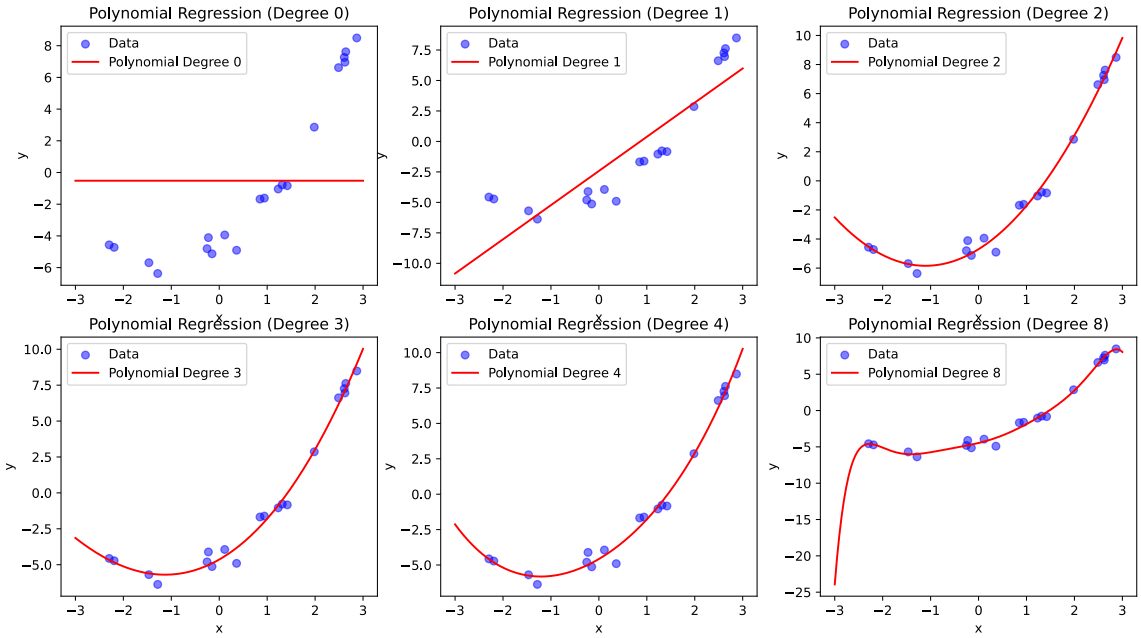
```
              Train       Valid           Test       TRVA         CV
Degree
0         18.459585   32.342363      22.100658  21.620221  28.779634
1          4.088535    7.127844       8.876307   9.934943   7.252893
2          0.218586    0.293732       0.245847   0.214016   0.321785
3          0.216819    0.283447       0.290080   0.275111   0.386635
4          0.118796    0.624726       0.969078   0.223993   0.641643
5          0.096532    0.573480       4.894836   1.039089   0.393977
6          0.007574    3.416787     213.297140   0.881471   0.399130
7          0.004999    6.862993    1261.988070   0.271719   1.738198
8          0.002082  401.651780  154266.871360  11.223554  25.777011
```

Polynominal with the lowest CV loss is the polynominal which probably generalizes the best. So that's the one to choose.

Task b

```
p = [0, 1, 2, 3, 4, 8]
x = np.linspace(start=-3, stop=3, num=256)
x = x.reshape(-1, 1)
x = pd.DataFrame(x, columns=['x'])
```

```
fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for i, deg in enumerate(p):
    row = i // 3
    col = i % 3
    ax = axes[row, col]

    model = models[deg]
    y_pred = model.predict(x)

    ax.scatter(X_trva, y_trva, color='blue', label='Data', alpha=0.5)
    ax.plot(x, y_pred, color='red', label=f'Polynomial Degree {deg}')
    ax.set_title(f'Polynomial Regression (Degree {deg})')
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.legend()
```



Task c

```
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error as mse
from sklearn.linear_model import Ridge

def rmse(y_true, y_pred):
    return np.sqrt(mse(y_true, y_pred))
```

```
train = pd.read_csv("train_real.csv")
test = pd.read_csv("test_real.csv")

X_train = train.drop(columns=['Next_Tmax'])
y_train = train['Next_Tmax']
X_test = test.drop(columns=['Next_Tmax'])
y_test = test['Next_Tmax']

train_loss = []
test_loss = []
cv_loss = []

models = {
    'Dummy': DummyRegressor(),
    'OSL Linear Regression': LinearRegression(),
    'Random Forest': RandomForestRegressor(),
    'SVR': Pipeline([('scaler', StandardScaler()), ('svr', SVR())]),
    'Ridge': Ridge()
}
```

```
def get_losses(model):
    model.fit(X_train, y_train)
    y_pred_test = model.predict(X_test)
    y_pred_train = model.predict(X_train)
    train_loss.append(rmse(y_train, y_pred_train))
    test_loss.append(rmse(y_test, y_pred_test))

    kf = KFold(n_splits=10, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train, y_train,
scoring='neg_root_mean_squared_error', cv=kf)
    cv_loss.append(-scores.mean())
```

```
for name, model in models.items():
    get_losses(model)
```

```python
results_df = pd.DataFrame({
    'Model': list(models.keys()),
    'Train RMSE': train_loss,
    'Test RMSE': test_loss,
    'CV RMSE': cv_loss
}).set_index('Model')

print(results_df)
```

```
                      Train RMSE  Test RMSE   CV RMSE
Model
Dummy                   3.089275   2.997846  3.080440
OSL Linear Regression   1.378378   1.483817  1.447004
Random Forest           0.534290   1.476183  1.416633
SVR                     1.406482   1.644513  1.727787
Ridge                   1.379322   1.484116  1.443108
```

1. The lowest test_RMSE is achieved by the Linear Regression model. So this is the best one for the given test set. However Random Forest has the best CV_RMSE, so it might generalize better to unseen data.
2. We can see that Generally Train RMSE is lower than Test RMSE, which is expected as models tend to perform better on the data they were trained on. CV RMSE is usually somewhere in between, as it estimates the performance better due to cross-validation.
3. We can achieve better performance by hyperparameter tuning, feature engineering, or trying more advanced models.

Problem 3

Task a

	Small Flexibility Models	Average Flexibility Models	High Flexibility Models
Training Error	Higher - model too simple to fit data	Lower - captures main structure	Lowest - can fit almost perfectly
Test Error	Higher - underfitting	Lowest - best balance	Higher - overfitting
(squared) Bias	High - oversimplified assumptions	Medium	Low - fits data closely
Variance	Low - stable predictions	Medium	High - sensitive to data noise

	Small Flexibility Models	Average Flexibility Models	High Flexibility Models
irreducible Error	Constant - noise in data cannot be reduced by any model	Constant	Constant

Task b

i

```
def f(x):
    return -2 - x + 0.5 * x**2

np.random.seed(42)

epsilon_std = 0.4
epsilon_mean = 0

training_size = 1000
set_size = 10
training_sets = []

results = pd.DataFrame(columns=['Degree', 'Irreducible', 'BiasSq', 'Variance',
'Total', 'MSE']).set_index('Degree')
```

```
def irreducible_error(y0, f0, y_pred):
    return np.mean((y0 - f0) ** 2)

def bias_squared(y0, f0, y_pred):
    return (np.mean(y_pred) - f0) ** 2

def variance(y0, f0, y_pred):
    return np.mean((y_pred - np.mean(y_pred)) ** 2)

def Total_error(y0, f0, y_pred):
    return irreducible_error(y0, f0, y_pred) + bias_squared(y0, f0, y_pred) +
variance(y0, f0, y_pred)

def MSE(y0, y_pred):
    return np.mean((y0 - y_pred) ** 2)
```

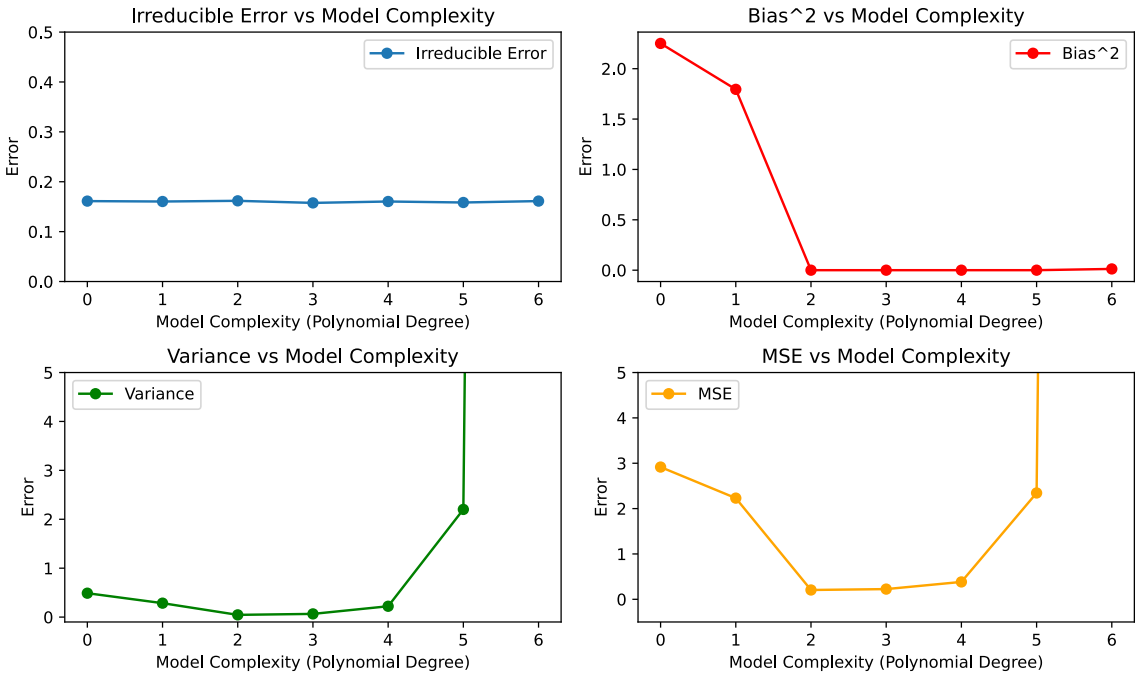
```
def make_poly_ols(degree: int) -> Pipeline:
    include_bias = True if degree == 0 else False
    return Pipeline([
        ("poly", PolynomialFeatures(degree=degree,
```

```
include_bias=include_bias)),
        ("scaler", StandardScaler()),
        ("ols", LinearRegression())
    ])

for i in range(training_size):
    for j in range(set_size):
        epsilon = np.random.normal(loc=epsilon_mean, scale=epsilon_std,
size=set_size)
        x = np.random.uniform(-3, 3, set_size)
        y = f(x) + epsilon
        training_sets.append((x, y))

degrees = np.arange(0, 7)
models = {}
for degree in degrees:
    model = make_poly_ols(degree)
    model.fit(x.reshape(-1, 1), y)
    models[degree] = model



for degree, model in models.items():
    stats = []
    for train_set in training_sets:
        x_train, y_train = train_set
        model.fit(x_train.reshape(-1, 1), y_train)
        test_x = 0
        test_y = f(test_x) + np.random.normal(loc=epsilon_mean,
scale=epsilon_std)
        f0 = f(test_x)
        y_pred = model.predict(np.array([[test_x]]))
        stats.append((f0, y_pred[0], test_y))

    f0_vals = stats[0][0]
    y_pred_vals = np.array([stat[1] for stat in stats])
    y_vals = np.array([stat[2] for stat in stats])

    irr = irreducible_error(y_vals, f0_vals, y_pred_vals)
    bias_sq = bias_squared(y_vals, f0_vals, y_pred_vals)
    var = variance(y_vals, f0_vals, y_pred_vals)
    total = Total_error(y_vals, f0_vals, y_pred_vals)
    mse_val = MSE(y_vals, y_pred_vals)

    results.loc[degree] = [irr, bias_sq, var, total, mse_val]

print(results)
```

```
        Irreducible    BiasSq    Variance       Total         MSE
Degree
0          0.161181  2.250944    0.488448    2.900572    2.915900
1          0.160379  1.795062    0.284751    2.240192    2.230935
2          0.161755  0.000006    0.045453    0.207214    0.205366
3          0.157590  0.000005    0.065332    0.222926    0.225011
4          0.160392  0.000009    0.221433    0.381834    0.383193
5          0.158406  0.000054    2.201312    2.359772    2.344425
6          0.161227  0.012442  146.532829  146.706498  146.593247
```

ii

```
x = degrees
irr = results['Irreducible']
bias_sq = results['BiasSq']
var = results['Variance']
mse = results['MSE']

fig, axes = plt.subplots(2, 2, figsize=(10, 6))


axes[0, 0].plot(x, irr, label='Irreducible Error', marker='o')
axes[0, 0].set_ylim(0.0, 0.5)
axes[0, 0].set_title('Irreducible Error vs Model Complexity', )
axes[0, 1].plot(x, bias_sq, label='Bias^2', marker='o', color='red')
axes[0, 1].set_title('Bias^2 vs Model Complexity')
axes[1, 0].plot(x, var, label='Variance', marker='o', color='green')
axes[1, 0].set_title('Variance vs Model Complexity')
axes[1, 0].set_ylim(-0.10, 5.0)
axes[1, 1].plot(x, mse, label='MSE', marker='o', color='orange')
axes[1, 1].set_title('MSE vs Model Complexity')
axes[1, 1].set_ylim(-0.50, 5.0)

for ax in axes.flat:
    ax.set_xlabel('Model Complexity (Polynomial Degree)')
    ax.set_ylabel('Error')
    ax.legend()

fig.tight_layout()
```



iii

The terms behave as expected: - Irreducible Error remains constant across all model complexities - Bias^2 decreases as model complexity increases - Variance increases with model complexity - MSE initially decreases, reaches a minimum, and then increases again, illustrating the bias-variance trade-off.

Problem 4

Problem 5

Task a

```
d1 = pd.read_csv("d1.csv")
d2 = pd.read_csv("d2.csv")
d3 = pd.read_csv("d3.csv")
d4 = pd.read_csv("d4.csv")
```

```
models = {}
slopes = {}
intercepts = {}
rsquared = {}
for i, data in enumerate([d1, d2, d3, d4], start=1):
    model = LinearRegression()
    model.fit(data[['x']], data['y'])
```

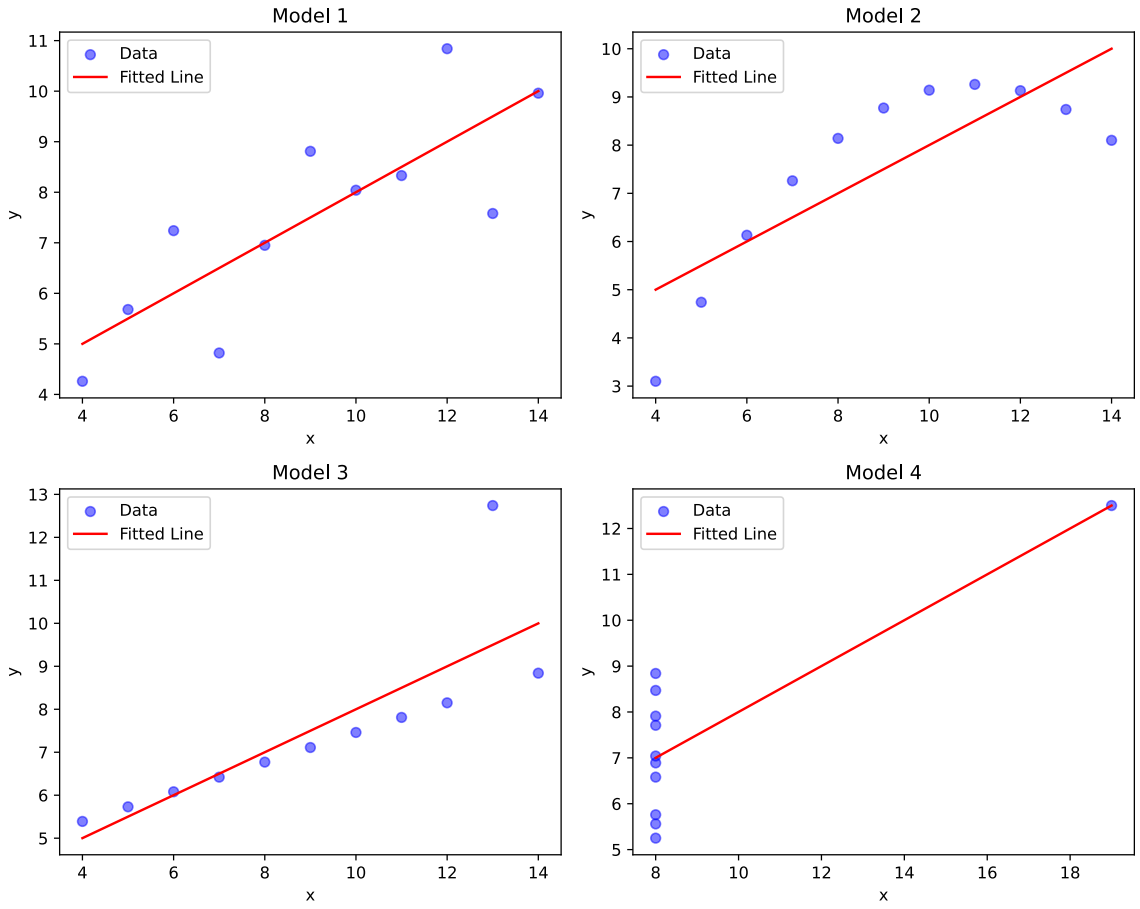
```
    models[f'model_{i}'] = model
    slopes[f'model_{i}'] = model.coef_[0]
    intercepts[f'model_{i}'] = model.intercept_
    rsquared[f'model_{i}'] = model.score(data[['x']], data['y'])

results_df = pd.DataFrame({
    'Model': list(models.keys()),
    'Slope': list(slopes.values()),
    'Intercept': list(intercepts.values()),
    'R^2': list(rsquared.values())
}).set_index('Model')

print(results_df)
```

```
            Slope  Intercept       R^2
Model
model_1  0.500091   3.000091  0.666542
model_2  0.500000   3.000909  0.666242
model_3  0.499727   3.002455  0.666324
model_4  0.499909   3.001727  0.666707
```

it's not safe to conclude that even with positive slopes the x and y are positively correlated in all cases, becuase the linear regression only captures the trend of the data and not the actual distribution.

Task b

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for i, (data, model) in enumerate(zip([d1, d2, d3, d4], models.values()),
start=1):
    row = (i - 1) // 2
    col = (i - 1) % 2
    ax = axes[row, col]
    ax.scatter(data['x'], data['y'], color='blue', label='Data', alpha=0.5)
    x_range = np.linspace(data['x'].min(), data['x'].max(), 100).reshape(-1,
1)
    x_range_df = pd.DataFrame(x_range, columns=['x'])
    y_pred = model.predict(x_range_df)
    ax.plot(x_range, y_pred, color='red', label='Fitted Line')
    ax.set_title(f'Model {i}')
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.legend()
plt.tight_layout()
```



While the fitted lines looks similar across all four datasets, the underlying data distributions are quite different. So it's not safe to conclude that even with positive slopes the x and y are positively correlated in all cases. This is clearly shown in dataset 2.

Task c

1. Dataset 1:
   - Non
2. Dataset 2:
   - Non-linearity of the data